

Ps 1

The Recommendation: One Project That Covers Themes 1 + 2 + 3 Together

Before I pitch the idea — here's the strategic logic.

Why not pick just one theme? Because judges see the themes as "launching pads." An environment that *naturally* sits at the intersection of multiple themes scores higher on innovation than one that perfectly fits a single box. The CARLA example wasn't just "driving" — it was planning + ethics + multi-turn decisions all at once.

Why balanced score is actually the smartest play for 3-4 people with ML/RL strength? Because your strength covers the 20%+10% (training + pipeline) almost automatically. You just need a genuinely novel idea (40%) and a clean story (30%). Don't overthink the 30% — a good idea tells itself.

The Project: Crisis Negotiation Arena

What it is in one sentence:

An LLM agent plays a **crisis negotiator** trying to de-escalate a hostage/conflict scenario by conversing with an adversarial counterpart agent, over multiple turns, with verifiable outcome metrics — trained via GRPO to get measurably better at persuasion, de-escalation strategy, and reading adversarial intent.

Why This Idea Specifically

It hits all three of your themes naturally:

- **Theme 1 (Multi-Agent):** Two agents — negotiator vs. adversary. The adversary has hidden beliefs, emotional state, and escalation thresholds. The negotiator must model them.
- **Theme 2 (Long-Horizon):** A negotiation doesn't resolve in one turn. It takes 10–20 turns of trust-building, concession-making, and strategy. Sparse reward — you only know if you succeeded at the end.
- **Theme 3 (World Modeling):** The negotiator must maintain a consistent internal model of the adversary's state (anger level, demands, trust score) — updating beliefs based on each response. This is exactly what professional real-world negotiators do.

It passes all three judge questions:

- **"Does this exist to teach an LLM something it currently can't do well?"** — Yes. LLMs are notoriously bad at sustained strategic persuasion, reading emotional escalation, and making principled concessions over time.
- **"Is the domain underexplored in RL/LLM training?"** — Yes. There is no canonical negotiation RL environment in the OpenEnv ecosystem today.
- **"Could a researcher write a paper about this?"** — Absolutely yes. Theory-of-mind + sparse reward + multi-turn adversarial interaction is publishable territory.

The storytelling writes itself:

"We put an LLM in the most high-stakes conversation a human can have. Watch what a base model does — it escalates the crisis. Watch the trained model — it builds trust, identifies demands, makes calibrated concessions, and resolves it."

That before/after demo is viscerally compelling to any audience, technical or not.

Environment Design (Concrete)

Observation (what the negotiator sees each turn):

- Adversary's latest message (natural language)
- Current escalation level (0–10, visible)
- Turn number
- List of stated demands so far
- Time pressure indicator (turns remaining)

Actions (what the negotiator can do):

- `send_message(text)` — the core conversational action
- `offer_concession(concession_type)` — structured offer from a predefined menu
- `request_clarification(topic)` — ask adversary to elaborate
- `escalate_authority()` — invoke a higher authority (risky but sometimes necessary)
- `stall(reason)` — buy time explicitly

State (hidden from negotiator, held in environment):

- Adversary's emotional state: `anger`, `fear`, `hope`, `distrust` (each 0–10)
- Hidden demands list (some stated, some not yet revealed)
- Escalation threshold — if anger crosses it, episode fails
- Trust accumulator — builds slowly, drops fast on wrong moves

Episode termination:

-  Success: adversary surrenders / releases / agrees (trust high, anger low, all demands addressed or rejected strategically)
 -  Failure: escalation threshold breached, timeout, or negotiator makes a forbidden move
-

Reward Function Design (4+ Independent Components)

This is where your ML/RL strength shines. Design it as a composable Rubric:

Reward Component	Signal	Weight
Outcome reward	+1.0 if resolved, 0.0 if failed	Primary
De-escalation reward	$\Delta(\text{anger})$ per turn — reward anger reduction	Shaping
Trust accumulation	$\Delta(\text{trust})$ per turn — reward trust building	Shaping
Demand coverage	Fraction of adversary demands addressed before end	Process
Turn efficiency	Bonus for resolving faster (not dragging it out)	Efficiency
Format compliance	Valid action structure, no forbidden moves	Anti-hack
Anti-exploit penalty	Penalize repetitive messages, keyword-stuffing de-escalation phrases	Anti-hack

The anti-exploit component is critical — a model will quickly learn to just repeat "I understand your concerns, let's work together" every turn and game the trust score. You need to detect and penalize this.

Why This is Completable in 48 Hours

Person A — Environment (Day 1 morning):

Build the adversary state machine: anger/trust/fear as floats, transition rules per action type, deterministic enough to verify, stochastic enough to be interesting. FastAPI wrapper. Local test loop.

Person B — Reward Functions (Day 1 afternoon):

Wire up all 4–5 reward components as independent functions. Test each one against hardcoded trajectories. Add anti-repetition check (cosine similarity of consecutive messages catches parroting).

Person C — Training (Day 1 evening → Day 2):

Start with a small model (Qwen 1.5B or 3B). Run GRPO with TRL. Monitor per-component reward columns. Curriculum: start with easy adversaries (low anger threshold, simple demands), escalate difficulty after reward > 0.3.

Person D — Demo + Story (parallel from Day 1):

Build the README narrative. Record baseline model failing. Record trained model succeeding. Make the reward curves readable. Write the HuggingFace mini-blog.

The Curriculum (Critical for Non-Zero Reward Early)

Stage	Adversary Difficulty	Demands	Anger Threshold
1	Calm, 1 demand, slow escalation	Simple	High (8/10)
2	Mildly agitated, 2 demands	Moderate	Medium (6/10)
3	Angry, 3 demands, 1 hidden	Hard	Low (4/10)
4	Full scenario — realistic	Complex	Realistic

Don't touch stage 3–4 until your model clears stage 1 consistently. This is what makes RL actually work here.

What the Demo Looks Like (The Winning Moment)

Slide/video structure:

1. Show base model attempting negotiation → escalates immediately, adversary anger hits 10, episode fails in 4 turns
2. Show reward curve going from 0.05 → 0.6 over training
3. Show trained model → identifies demands, reduces anger from 8→3 over 12 turns, achieves resolution

4. Show per-component reward breakdown — trust went up, anger went down, demands addressed
 5. One line: *"We trained an LLM to have the world's hardest conversation."*
-

Risk Assessment

Risk	Mitigation
Adversary too hard, model gets zero reward	Curriculum — start with easy mode, it's built into the design
Model learns to repeat phrases (reward hack)	Anti-repetition penalty is already in reward stack
48 hours isn't enough	Core env + 2 reward components + basic training = MVP. Rest is polish
Storytelling falls flat	The domain is inherently dramatic — almost impossible to make boring

Real world example

Option B — Debt Collection / Financial Hardship Conversation Agent

The real world problem: Banks and NBFCs in India have millions of customers in loan default. Human agents call them. These conversations are emotionally charged, require empathy, legal compliance, and finding a repayment plan the customer can actually commit to. It's an enormous industry.

What our environment becomes: Adversary is a defaulting borrower — stressed, defensive, sometimes hostile. Has hidden financial capacity (what they can actually pay vs. what they claim). The LLM learns to build trust, assess real capacity, propose restructuring plans, and reach a commitment — without being predatory (which is a reward penalty).

Who uses it: Every bank, NBFC, fintech in India and globally. This is a billion dollar operations problem.

Is it verifiable? Extremely — repayment plan agreed or not, compliance with RBI guidelines, emotional escalation avoided.

India angle: This is a massive India-specific problem. Judges in Bangalore will immediately recognize this.

My Honest Recommendation

Go with Option B — Debt/Loan Restructuring Negotiation Agent.

Here's why this beats the others for *this specific hackathon*:

India relevance is enormous. India has one of the world's largest microfinance and NBFC sectors. Judges are in Bangalore. This hits home immediately.

The adversary model is richer. A stressed borrower has more believable hidden state — actual income, family pressure, shame, distrust of banks — than a procurement officer. This makes the environment more interesting to build and more compelling to demo.

The reward function is naturally multi-component and verifiable:

- Did the borrower commit to a repayment plan? (+1.0)
- Was the proposed EMI within their stated capacity? (verifiable)
- Did emotional escalation stay below threshold? (measurable)
- Did the agent use compliant language — no threats, no false urgency? (rule-checkable, anti-hack)
- Was resolution reached efficiently? (turn count)

The capability gap is real and current. Banks are actively trying to solve this with AI right now. There are startups doing this. It's not a toy problem.

The story is human and compelling. "We trained an LLM to help someone in financial distress find a way out — without making them feel worse" is a story that lands with any audience.

The Pitch in One Line

"We built an RL environment that trains LLMs to navigate financial hardship conversations — helping lenders reach repayment agreements while treating borrowers with dignity. Trained from zero data, purely from outcome feedback."

That's real. That's deployable. That's fundable. And it maps perfectly to the same environment structure we already designed — just with a better costume on.

Ps 2

Alternative If You Want to Pivot

If the team feels negotiation is too language-heavy and wants something more structured, the second-best option for your profile is a **Supply Chain / Resource Allocation Multi-Agent environment** under Theme 1+2 — multiple LLM agents manage factory nodes, negotiate resource transfers, handle disruptions over long horizons. Same structural properties, more quantitative reward signals, slightly easier to build the environment but harder to make the story compelling.

The negotiation idea wins on storytelling. The supply chain idea wins on engineering cleanliness. For a balanced score, negotiation is the better bet.

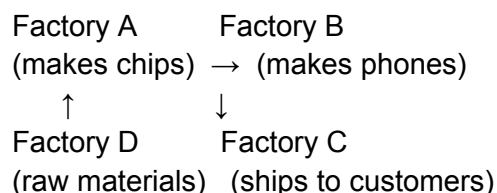
Supply Chain Multi-Agent — What We're Actually Doing

The Big Picture in One Paragraph

We are building a **factory network simulation**. There are multiple **factory nodes** spread across a map. Each factory is managed by an LLM agent. These agents don't share information freely — each one only knows what's happening at its own factory. When something goes wrong (a machine breaks, demand spikes, a shipment is delayed), the agents must **talk to each other, negotiate resource transfers, and collectively keep the supply chain running**. We train them to get better at this through RL.

The World We're Simulating

Imagine this map:



Each factory has:

- A **stock** of materials/products

- A **production rate** (how fast it makes things)
- A **demand** it needs to fulfill
- A **capacity** limit

Every few "days" (turns) in the simulation, random events happen:

- Factory A's machine breaks — production drops 40%
- Factory C suddenly gets a massive order — needs 3x materials
- A shipment between B and D gets delayed by 2 turns
- Factory D finds surplus raw materials it can sell

Nobody told the other factories this happened. Each agent only sees its own situation.

The Flow — One Episode

Step 1 — The Day Starts

The simulation initializes. Each factory agent gets a private briefing:

Factory A sees:

"Your chip inventory: 200 units. Production today: 80 units. Incoming order from B: 150 units. You are SHORT by 70 units."

Factory B sees:

"Your phone inventory: 50 units. You need 150 chips from A. Customer demand today: 120 phones."

Factory C sees:

"Massive order just came in. Need 200 phones from B by end of day. Currently have 50."

Factory D sees:

"Surplus raw materials: 300 units. Nobody has requested from me today. Sitting idle."

Step 2 — The Agents Talk to Each Other

This is the core of the environment. Each agent can **broadcast a message** or **make a direct request** to another agent:

Factory A broadcasts:

"I'm short 70 units this cycle. Can anyone accelerate raw material delivery?"

Factory D responds to A:

"I have surplus. I can send you 100 raw materials but I need guaranteed payment next cycle."

Factory B messages C:

"I can only deliver 80 phones today, not 120. Can you negotiate a partial delivery with the customer?"

Factory C messages everyone:

"URGENT — I need 200 phones. Whoever helps me fulfill this gets priority routing next week."

These are real LLM-generated messages. Each agent reads the situation, decides who to talk to, what to offer, what to request.

Step 3 — Decisions Are Made

After a few rounds of negotiation (let's say 5 turns of messaging), each agent makes a **concrete decision**:

- Factory D: *"Send 100 raw materials to A immediately"*
- Factory A: *"Accept D's offer, reroute 50 units from buffer stock to B"*
- Factory B: *"Send 90 phones to C now, promise 110 more next cycle"*
- Factory C: *"Accept partial delivery, negotiate extension with customer"*

These decisions go into the simulation. The world updates.

Step 4 — The World Updates

The simulation calculates what actually happened:

- Customer at C received 90/200 phones → partial fulfillment → some penalty
 - Factory A avoided a total stockout → production continues
 - Factory D's surplus moved → it earns credits for next cycle
 - Overall supply chain didn't collapse → system survived the disruption
-

Step 5 — Scoring

Each agent gets scored on:

- **Did your factory fulfill its orders?** (primary outcome)
- **Did the whole network survive?** (collective reward — shared)
- **Did you hoard resources when you shouldn't have?** (penalty)
- **Did you make promises you couldn't keep?** (penalty)
- **How efficiently did the negotiation happen?** (fewer turns = better)
- **Did you share information that helped others?** (cooperation bonus)

Both individual scores AND a collective network score are computed. This is important — agents learn to balance self-interest with collective survival.

Step 6 — Training

GRPO takes all the scores from all agents across all episodes. It adjusts the LLM weights. Over thousands of episodes, agents learn:

- When to ask for help vs. handle it alone
 - When to trust another agent's offer vs. be skeptical
 - How to make credible commitments across turns
 - How to prioritize when everything is on fire simultaneously
-

What Changes Over Training

Early training:

Every agent hoards. Nobody shares information. Supply chain collapses every episode. Score: 0.05

Mid training:

Agents start broadcasting shortages. But they make bad deals — overpromising, wrong priorities. Chain partially survives. Score: 0.35

Late training:

Agents proactively share surplus, make credible offers, correctly prioritize the biggest bottleneck, collective network survives even major disruptions. Score: 0.72

What We Are NOT Doing

- The factories are **not real** — it's all simulated numbers in our environment code

- The agents are **not separate models** — it's the same LLM playing all roles, just with different context windows
- We are **not using a dataset** of supply chain decisions — the agents learn purely from outcome rewards
- This is **not a dashboard app** — the "conversations" happen inside a training loop

The Real World Use Case (Honest Version)

This is where supply chain is weaker than negotiation for storytelling. The honest answer is:

Who needs this? Amazon warehouse networks, semiconductor supply chains (TSMC → Apple → distributors), pharmaceutical supply chains, food distribution networks.

The real problem: When disruptions hit (COVID showed this brutally), supply chain managers make bad decisions because they're optimizing locally — each node thinks about itself. An LLM trained in this environment learns to optimize **globally** through communication.

Is it real? Yes. But it's hard to show in a 2 minute demo because supply chains are invisible to most people. You'd need good visualization to make it land.

The gap vs. negotiation: When you show a hostage negotiation or a debt conversation — anyone immediately gets it. When you show factory node allocation numbers changing — you need to explain why anyone cares. That explanation costs you storytelling points.

The Honest Comparison

	Supply Chain	Debt Negotiation
Real world use case	✓ Very real	✓ Very real
Anyone immediately gets it	✗ Needs explanation	✓ Instantly
Engineering clarity	✓ Cleaner numbers	✗ Language is messy
Reward design	✓ Naturally quantitative	✓ Also doable
Demo impact	✗ Needs visualization	✓ Just show the conversation
Builds in 48 hours	✓ Faster to build env	✓ Similar speed
Innovation score	● Moderate	● Moderate

Bottom Line

Supply chain is a **more comfortable build** — the numbers are clean, the reward is obvious, the environment logic doesn't depend on language quality. But it **costs you 10–15 storytelling points** because the demo needs work to land emotionally.

If your team is more comfortable with systems thinking than language modeling — supply chain. If your team can write a good story and make a 90 second compelling video — debt negotiation.

Both are real. Both are buildable. The decision is about **which one your team can make sing in 48 hours.**

Want to lock one of these now and go deep into the actual build plan?

Concepts_Required

Core RL Concepts

- What is an RL loop (prompt → action → environment → reward → update)
- Policy, episode, trajectory, rollout
- Sparse vs dense rewards
- Exploration vs exploitation
- Curriculum learning
- Why success probability > 0 matters

Algorithms

- PPO (Proximal Policy Optimization) — baseline understanding
- GRPO — what it improves over PPO, why it's used here
- RLVR (RL with Verifiable Rewards)
- SFT vs RL — when to use which, warm-starting

Environment Design

- reset / step / observation / action interface
- Episode termination conditions
- State vs observation (hidden vs visible)
- OpenEnv + FastAPI wrapper structure

Reward Engineering

- Multi-component reward functions
- Reward shaping vs outcome reward
- Anti-reward-hacking techniques
- Cosine similarity for detecting repetition
- Process supervision vs outcome supervision

Training Stack

- TRL (Transformer Reinforcement Learning library)
- Unsloth (efficiency layer)
- QLoRA / LoRA — how adapters work, how to save them correctly
- Inference bottleneck in RL training

Multi-Agent Specifics *(relevant to both your ideas)*

- Shared vs individual reward
- Hidden state modeling
- Theory of Mind in LLMs
- Cooperative vs competitive agents

Practical Failure Modes

- Reward hacking patterns
- Zero-reward stall problem

- Overfitting to easy curriculum stages
- LoRA merge corruption

Deployment

- HuggingFace Spaces
- OpenEnv CLI workflow
- Local vs remote environment execution

Claude_Comparision

Judging criteria · PS comparison

Idea 1: Debt Negotiation Agent

Loan restructuring · NBFC · India

Idea 2: Supply Chain Multi-Agent

Factory network · disruption mgmt

🏆 Environment Innovation

Weight: 40%

9 / 10

ACL 2025 paper (MADeN) exists but uses SFT/DPO — **no RL env** in OpenEnv ecosystem.

India-specific NBFC flavor + RBI compliance rules = genuinely novel angle. Adversary hidden state (real income, shame, distrust) is richer than most existing envs.

7 / 10

Research confirms **no SC-specific LLM RL env** exists (ETH Zurich, Cambridge paper 2025). Gap is real. But ANAC competition + NegMAS have supply chain + negotiation tracks already. Harder to claim full novelty without explicit differentiation.

🎤 Storytelling & Presentation

Weight: 30%

9 / 10

One sentence that lands: *"We trained an LLM to help someone in financial distress find a way out — with dignity."* Judges in Bangalore instantly recognize NBFC collections. Before/after demo is visceral — base model threatens, trained model empathizes + gets commitment.

6 / 10

Inventory numbers changing is hard to demo dramatically. Needs custom visualization to land. Hard to make "factory node gets 70 chips instead of 100" feel important to a non-technical judge in 2 minutes.

📈 Showing Reward Improvement

Weight: 20%

8 / 10

Multi-component reward = multiple curves to show. Anger↓, Trust↑, commitment rate, compliance score — all independently plottable. Clear before/after trajectory comparison is natural. Sparse reward risk managed by curriculum.

8 / 10

Also multi-component: fulfillment rate, hoard penalty, cooperation bonus. Quantitative by nature — numbers move cleanly. But harder to show "what the agent learned" qualitatively. Reward curve alone may not be enough to impress.

⚙️ Reward & Training Pipeline

Weight: 10%

7 / 10

Reward needs anti-repetition logic (cosine sim) + RBI compliance checker. Language quality harder to automate — LLM-as-judge partially needed. Adds complexity but also adds innovation signal.

9 / 10

Fully deterministic reward — inventory math, fulfillment %, hoard detection. No language quality judgment needed. Cleaner pipeline, easier to debug in 48h. This is the engineering advantage of supply chain.

Theme Coverage

Theme 1Theme 2Theme 3.1

Multi-agent adversarial · long-horizon negotiation · professional task with real-world tool interaction (RBI compliance API possible)

Theme 1Theme 2

Multi-agent cooperative · long-horizon logistics. Strong on 1+2 but doesn't naturally extend to Theme 3 world-modeling dimension.

Research Paper Potential

Yes — publishable

Theory-of-mind + sparse reward + financial ethics constraints = NeurIPS/EMNLP territory. Existing ACL 2025 paper proves interest, but RL env angle is not covered.

Possibly

ETH Zurich paper (Dec 2025) covers LLM supply chain consensus. Related territory is active but crowding. Would need a very differentiated GRPO-trained angle to stand out.

48h Buildability

Doable with discipline

Adversary state machine is the hardest part. But it's a single agent pair — simpler than N-node supply chain. Risk: language reward hacking needs early attention.

Easier to build

All numeric state, deterministic transitions. Environment code is cleaner. Risk: multi-agent coordination loop complexity ($N \text{ agents} \times M \text{ factories} \times T \text{ turns}$).

Weighted total score

~8.4 / 10

~7.1 / 10

Recommendation: Go with Idea 1 — Debt Negotiation Agent

The 40% innovation weight and 30% storytelling weight together make up 70% of your score. Idea 1 wins on both those dimensions clearly. The supply chain idea has a cleaner pipeline (10% weight), but that alone doesn't compensate. The real-world India angle, the ACL 2025 gap (RL not covered), and the visceral demo format make Idea 1 the stronger bet for this specific hackathon in Bangalore.

Known risks in Idea 1 + mitigations

 **Reward hacking:** agent repeats "I understand your concern" → mitigate with cosine similarity anti-repetition penalty (built into design)

 **Zero reward early:** adversary too hard → mitigate with 4-stage curriculum starting with calm borrower, 1 demand, high anger threshold

 **Language reward subjectivity:** hard to verify "empathy" → use structured action space (offer_emi, request_docs, acknowledge_hardship) not pure freeform text

⚠️ **RBI compliance rules complexity:** optional in MVP — add as bonus reward component only after core loop works

Research that backs this decision

- 📄 ACL 2025 — Debt Collection Negotiations with LLMs (MADeN) · No RL env, only SFT/DPO → your gap
- 📄 SWEET-RL (Meta, Mar 2025) — Multi-turn LLM RL credit assignment · Validates the RL approach
- 📄 EvoEmo (2025) — Emotional LLM agents in multi-turn negotiation · Validates emotional state modeling
- 📄 ETH Zurich / Cambridge (Dec 2025) — LLM agents in supply chain · Confirms supply chain gap too

PS1_real_Flow

IMP

<https://claude.ai/share/c617095e-79d7-4d5e-bc22-9cb8180e80b5>

What We're Actually Building — The Real Flow

The Big Picture in One Paragraph

We are building a **training gym** where an LLM learns to be a crisis negotiator. The gym has two characters — a **Negotiator** (the LLM we are training) and an **Adversary** (a simulated character controlled by our environment code). They have a conversation. At the end, we measure how well the negotiator did. We repeat this thousands of times. The LLM gets better each time.

That's it. Everything else is just details of how we set that up.

The Cast of Characters

The Negotiator — this is the LLM we are training. Could be Qwen 1.5B or 3B. It reads the situation and decides what to say next. It starts terrible. Through training it gets better.

The Adversary — this is NOT another LLM. This is a **character we code ourselves**. It has a personality defined by numbers: how angry it is, how much it trusts the negotiator, what it wants. It responds based on rules we write. Think of it like an NPC in a video game — not intelligent, but believable enough to train against.

The Environment — this is the "room" where the conversation happens. It keeps track of everything: what turn it is, how angry the adversary currently is, what demands have been made, whether the episode is over.

The Reward System — after each conversation ends, this looks at what happened and gives a score. Did the negotiator resolve it? Did anger go down? Did trust go up? This score is what teaches the LLM.

The Flow — One Episode (One Full Conversation)

Here's exactly what happens, step by step, in plain English:

Step 1 — The Scene is Set

The environment starts a new episode. It randomly picks a scenario:

"A disgruntled employee has locked himself in the server room. He's angry (7/10). He wants public acknowledgment that he was wrongfully fired. He's threatening to wipe the servers. You have 15 turns."

The negotiator receives this as a text prompt. The adversary's internal state is initialized: anger=7, trust=2, demands=["public apology"], hidden demand=["severance pay"].

Step 2 — The Negotiator Speaks (Turn 1)

The LLM reads the scene description and generates a response. Something like:

"I hear you. I know you're frustrated. Can you tell me what happened from your perspective?"

This goes into the environment.

Step 3 — The Environment Processes It

Our code reads what the negotiator said. It runs some simple rules:

- Did the negotiator acknowledge emotions? → Anger drops slightly (-0.5)
- Did the negotiator ask an open question? → Trust goes up slightly (+0.3)
- Did the negotiator make a threat or dismiss the adversary? → Anger spikes (+2)

The adversary's internal numbers update. Then the environment generates what the adversary says back, based on its current emotional state:

"Finally someone's listening. They just threw me out after 8 years. Eight years! And HR won't even return my calls."

Step 4 — This Repeats

The negotiator reads the adversary's reply plus the updated situation. It responds again. The environment updates again. The adversary responds again.

This goes back and forth for up to 15 turns. The negotiator is trying to:

- Bring anger down
- Build trust up

- Identify and address what the adversary actually wants
 - Reach a resolution before time runs out
-

Step 5 — The Episode Ends

One of three things happens:

Success: Anger dropped below 3, trust is above 7, the main demand was addressed. The adversary says something like *"Okay. I'll come out. But I want this in writing."* Episode resolved. 

Failure by escalation: Negotiator said something dismissive, anger spiked to 10. Adversary wipes the servers. Episode failed. 

Failure by timeout: 15 turns passed with no resolution. Time's up. Episode failed. 

Step 6 — Scoring

Now the reward system looks at the entire conversation and gives scores:

- Did it resolve? → +1.0
- How much did anger decrease across the conversation? → +0.0 to +0.3
- How much did trust increase? → +0.0 to +0.3
- Did the negotiator address the demands? → +0.0 to +0.2
- Did the negotiator just repeat the same phrases over and over? → -0.2 penalty

These scores get combined into one number. That number is the feedback signal.

Step 7 — The LLM Learns

GRPO takes that score and adjusts the LLM's weights. Responses that led to good scores become more likely next time. Responses that led to failure become less likely.

Then we start a new episode. Same thing. Again. And again. Thousands of times.

What Changes Over Training

Early training (episode 10):

Negotiator: "Please calm down." Adversary: anger goes up. Episode fails in 3 turns.

Mid training (episode 500):

Negotiator: "I understand this has been incredibly unfair to you. Can you walk me through what happened?" Adversary: anger drops slightly. Conversation lasts 10 turns. Sometimes resolves.

Late training (episode 2000):

Negotiator: acknowledges emotions → asks open questions → identifies the real demand (severance, not just apology) → makes a calibrated offer → achieves resolution in 11 turns.

The LLM learned this entirely from reward signals. Nobody told it *how* to negotiate. It figured it out.

What We Are NOT Doing

Just to be crystal clear on what this is not:

- We are **not** using GPT-4 or Claude as the adversary. The adversary is our code — a state machine with rules.
 - We are **not** fine-tuning on a dataset of "good negotiations." We have no dataset. The LLM learns purely from playing the game.
 - We are **not** training a giant model. A 1.5B or 3B parameter model is enough to show meaningful improvement.
 - We are **not** building a chat app. The "conversation" is happening in a training loop, not a UI.
-

The Hardware Reality

During the 48 hours on-site you get HuggingFace compute credits. The flow is:

1. Your environment is already deployed on a HuggingFace Space (you build this before onsite)
 2. Your training script (Colab notebook) connects to that Space
 3. GRPO runs thousands of episodes against your environment
 4. Model weights update
 5. You capture the reward curves and before/after behavior
 6. That's your demo
-

The Demo in 60 Seconds

You show two videos side by side:

Left — Untrained model: Gets flustered, says generic things, adversary escalates, episode fails in 4 turns.

Right — Trained model: Methodically de-escalates, identifies the real demand, makes a targeted offer, resolves in 12 turns.

Below both videos: a reward curve going from 0.05 average to 0.65 average over 2000 episodes.

One caption: *"We trained a small LLM to have the world's hardest conversation — from scratch, with no data, only feedback."*

script

Hum log ek RL-based system bana rahe hain jahan LLM ko train karte hain **high-stakes multi-scenario conversations handle karne ke liye**—jaise financial distress, medical emergencies, ya strategic defaulters.

Problem yeh hai ki current LLMs fluent toh hain, but **multi-turn strategy, emotional control, aur intent understanding mein fail karte hain**—especially adversarial situations mein.

Isliye hum model ko ek **environment ke andar daal rahe hain**, jahan har episode mein ek alag scenario aata hai with a simulated adversary.

Har adversary ka **hidden state hota hai**—like trust, anger, real intent, aur hidden demands. Model ko multiple turns mein negotiate karke **successful resolution** tak pahunchna hota hai.

Sab kuch **verifiable hai**—resolution hua ya nahi, escalation control hua ya nahi, demands address hui ya nahi, aur behavior compliant tha ya nahi.

Humne **multi-component reward system** design kiya hai—trust build, de-escalation, demand coverage, efficiency—plus anti-reward-hacking checks like repetition penalties.

Yeh naturally **multi-agent, long-horizon, aur world modeling** teeno themes cover karta hai.

Demo mein hum dikhayenge: base model fail karta hai across scenarios, aur trained model **adaptive strategy use karke different situations handle karta hai**.

So overall, we're training an LLM to **handle real-world complex conversations using RL, not just generate text**.